

1. Rule-Based Trip Planner

- **What & Why:** Encodes itinerary “logic” with simple if/else rules (e.g. if budget is low → suggest hostels, if travel mode is train → find nearest station). It’s like a decision tree tailored to user inputs. This is useful as a starting point: no ML needed and behavior is predictable. For example, you might hardcode that if a user likes museums, add a museum visit.
- **How to Build:** Write Python functions (in Flask) that take user inputs (budget, duration, mode) and pick options from your database (events, hotels, attractions). You can store simple rules in JSON or use a Python rule-engine library (like [durable rules](#), open source). For example, define budget categories (“low, medium, high”) and corresponding hotel tiers. Use Python packages to fetch distances or travel info (e.g. [geopy](#)). No special APIs are required – you use your own data or free data (OpenStreetMap for location lookups).
- **Pros:** Very simple and transparent. Easy to implement and debug. No external services or costs. Gives full control over what recommendations look like.
- **Cons:** Inflexible and hard to scale. Rules can’t cover all user scenarios, and the system can’t “think outside the box.” In practice, large travel sites found that rule-based models “struggled to capture the nuances of user intent” ¹. As preferences grow, the rule set becomes complex.

2. AI Chatbot (Large Language Model)

- **What & Why:** Uses a conversational AI (like GPT-3.5/GPT-4 or an open LLM) to **generate** a travel plan in natural language. The user can chat with it (e.g. “I have \$1000, 3 days in Paris, want comfort, avoid flights”) and it returns a suggested itinerary. LLMs understand nuanced requests (“make it adventurous but cheap”) and can fuse all info into a coherent plan. This is useful for a **flexible, natural dialog**: the bot can ask clarifying questions and adjust the plan on the fly.
- **How to Build:** Use an AI API or open-source model. For example, use OpenAI’s GPT-3.5/GPT-4 via their API (there is a free trial credit) or an open model (LLaMA, Mistral, etc.) via Hugging Face Transformers. You’d craft a prompt that includes the selected event and user prefs, then generate an itinerary. For easier chaining or memory, you can use [LangChain](#) (free, MIT license) to manage prompts and context. Another option is a free chatbot framework like [ChatterBot](#) (pure Python) with a custom prompt backend. For deployment, you could host on Hugging Face Spaces or run a local model (small ones like [Mistral 7B](#) or Meta’s [Llama 2](#)). Many tutorials (e.g. Analytics Vidhya’s travel chatbot guide) show using Hugging Face and LangChain to create such assistants ² ³.
- **Pros:** Very flexible and user-friendly. Handles natural language and can offer creative suggestions (hotels, sights, alternate routes). Easy to prototype with free/open models. Can operate like a “smart friend” making suggestions.
- **Cons:** Can be unpredictable or “hallucinate” wrong info. It may suggest nonexistent hotels or inaccurate times unless grounded with real data. Using OpenAI’s API beyond the free trial involves cost. Running large models locally requires powerful hardware or GPUs. You’ll need to manage prompt engineering carefully. The output may be too general unless you guide it with structured data (e.g. by giving it real hotel names or transportation times).

3. Rasa Conversational Agent

- **What & Why:** Rasa is an open-source framework for building **contextual chatbots**. Unlike a raw LLM, Rasa lets you define intents (e.g. “provide budget”, “choose hotel”) and custom actions (e.g. call a flight search). It excels at structured dialogues (“slot filling”), so it can systematically ask follow-up questions: “What’s your budget? Great, what kind of accommodation?” and then build the itinerary step by step. This can make the trip planning feel like a guided conversation.
- **How to Build:** Use [Rasa Open Source](#), a Python library under Apache-2.0 license ⁴. You define training data (example user messages for each intent), and responses or actions for the bot. For example, create an intent “inform_budget” with training phrases (“I have 500 dollars”, “My budget is low”), and slot “budget.” Then create an action “action_plan_trip” that gathers all slots (budget, mode, duration, event) and returns an itinerary (you write the Python logic for this). Rasa has built-in NLU and dialogue management; you can run it on your server (it’s free). A Rasa agent can call Python code to query your MongoDB (for events/hotels) or call external APIs if needed.
- **Pros:** Completely free and runs locally. Great for multi-turn conversations and validations (it won’t miss a question). It’s more controlled than a free-form LLM, so results are predictable. You have full control of the dialog flow and can ensure all preferences are asked. Rasa has good documentation and community support.
- **Cons:** Higher learning curve (you must learn Rasa’s NLU, stories, slots). Setting up intents and training requires effort. Out-of-the-box it won’t “imagine” creative suggestions – you have to define responses or code how to pick them. It’s less flexible than a generative LLM: it won’t propose a hotel unless you’ve coded that path. It also won’t autonomously search the web (unless you program an API call).

4. Itinerary Optimizer (Operations Research)

- **What & Why:** Formulate the trip as an optimization problem (like a Traveling Salesperson or route-planning problem). For example, given a list of candidate hotels, attractions, and transport options, find the cheapest or fastest route that meets user constraints (duration, budget). This can automatically compute an efficient sequence of stops. Google’s OR-Tools library is ideal here: it can solve routing, TSP, and scheduling problems ⁵. This is useful when you have a fixed set of possible places and want the **optimal** path or schedule through them (e.g. visiting multiple attractions in order).
- **How to Build:** Use [OR-Tools](#) (free, Apache 2.0) in Python. For a simple case: list the locations (event venue, chosen hotel, places to eat, sightseeing spots) as nodes with coordinates. OR-Tools can solve the Travelling Salesperson Problem (TSP) or Vehicle Routing Problem (VRP) for you ⁶ ⁷. You encode constraints like “arrive by event start time” or “max cost = budget.” The output is an ordered itinerary (maybe with timings). You may combine OR-Tools with [NetworkX](#) or [OSMnx](#) (for calculating distances on real maps). All these tools are free Python libraries.
- **Pros:** Rigorous and produces an optimal or near-optimal plan under given constraints. Good for technical exercises (students learn algorithms). Once set up, it systematically handles travel times/costs. No API keys needed (you just need geodata, e.g. from OpenStreetMap).
- **Cons:** Doesn’t handle fuzzy preferences well (e.g. “adventurous” or “cozy”). Modeling non-numeric preferences (like “scenic route” vs “fastest route”) is hard. It also requires you to have or gather data (hotel locations, attraction lists, transit times), which can be complex. Solving large routing problems can be slow. It’s also not conversational – the user isn’t chatting, they just get a computed plan.

5. Data-Driven Planning via APIs (OpenTripMap, OSM, etc.)

- **What & Why:** Use open or free APIs to fetch real travel data (places, transit, flights) and assemble a plan. For example, use the [OpenTripMap API](#) to find points of interest and hotels near the event, or use a free flight search API (like Aviationstack or Kiwi's Tequila). This approach makes the plan **data-rich and realistic**, because you're pulling live info (open-source data) rather than hardcoding options.
- **How to Build:** Combine several free resources:
- **OpenTripMap (POIs):** This service lets you search millions of attractions and hotels worldwide ⁸ ⁹. It uses open data (OpenStreetMap, Wikimedia) under the hood, so no proprietary lock-in ⁹. You can get an API key (free tier) and query it for, say, "top museums near location X" or "hotels within 5 km".
- **OpenStreetMap (Maps/Routing):** Use the [OSM Nominatim API](#) for geocoding (converting place names to coordinates), and [OSRM](#) or [GraphHopper](#) for routing (car/bike/walk directions) if needed.
- **Flight/Train APIs:** There are some free or freemium APIs for transport. For example, [Aviationstack](#) offers free flight data (with signup). [Skyscanner's Tequila API](#) has a free tier for flight search.
- **Combine in Flask:** In your Flask app, write functions to call these APIs (using `requests` or a Python client). For instance, get the event location from your DB, ask OpenTripMap for recommended attractions, use OSRM to find travel times, and compute options under the user's budget.
- **Pros:** Access to **real** travel data. More dynamic and up-to-date than hardcoded rules. APIs like OpenTripMap are free and community-driven ⁹. You can provide maps and actual names, dates, prices. Many providers offer a generous free tier or trial.
- **Cons:** Integration overhead – learning different APIs and handling API keys/quotas. Not everything is free: flight/hotel booking APIs often have strict limits or are paid. Results depend on external service availability (rate limits, outages). Also you're not "creating" logic – you're just querying data, so you may still need some code to assemble it into an itinerary format.

6. OpenTripPlanner (Multi-Modal Itinerary Engine)

- **What & Why:** OpenTripPlanner (OTP) is an **open-source trip planning engine** originally designed for public transit, but it can combine transit, walking, biking, and driving ¹⁰. It builds a graph from OpenStreetMap (and optional transit GTFS data) and finds multi-modal routes. This is great if you want realistic door-to-door itineraries (e.g. "walk to train, take subway, then bike to event").
- **How to Build:** OTP is a Java server you run (it's free and open source). You would download GTFS feeds for your area (transit schedules) and use OpenStreetMap for the street network. Launch the OTP server locally or on a free-tier cloud. Then from Flask, you make HTTP requests to OTP's API to get routes (it outputs JSON with steps and times). You can use the [OTP REST API](#) directly or via Python libraries like [pyotp](#) (if available).
- **Pros:** Very realistic routes and time estimates for complex, city-wide trips. Supports combining modes (bus, train, walk). It's entirely free and leverages community data (OSM and GTFS). Once set up, it's extremely reliable.

- **Cons:** Heavy setup – you need to install Java, obtain GTFS data, and configure OTP. It's more complex than a simple Python script. If you only need point A to event location, it's overkill. Also, OTP itself doesn't incorporate business logic or preferences (you'd have to build on top of its output).

7. Static Itinerary Templates (Baseline Approach)

- **What & Why:** Create a library of **fixed itinerary templates** or “wizards” based on event types. For instance, a music festival template might always include “Day 1: Arrive and hotel check-in; Day 2: Festival; Day 3: City tour.” You then fill in details (which hotel, which attractions) using user inputs. This isn't AI, but it's a practical way to give users an instant plan without complex logic. It's useful as a fallback or minimum viable feature.
- **How to Build:** In your database or config, define a few common event categories (concert, conference, sightseeing trip) and corresponding day-by-day outlines. When a user selects an event and preferences, pick the right template and populate it. For example, if the budget is low, the template might say “Stay at Hostel X”; if high, “Stay at Hotel Y.” You can store these templates as JSON in MongoDB and render them in your Flask views. No external tools needed.
- **Pros:** Easiest to implement. Guaranteed to produce a coherent itinerary (it's pre-written). No AI unpredictability. Useful when you have very limited data or time.
- **Cons:** Not personalized or dynamic – the output is largely static and may feel generic. It doesn't scale well to unique requests. Essentially, it's a hardcoded plan rather than a “smart” one. Still, it can provide a quick stopgap solution while developing more advanced methods.

Each of these approaches can be combined or iterated on. For example, you might start with rule-based logic or static templates, then add an LLM or Rasa agent to enhance it. All the tools mentioned above are free or have free tiers (e.g. Rasa is open-source ⁴, OR-Tools is free ⁵, OpenTripMap uses open data ⁹, OpenAI provides trial credits, etc.), making them suitable for a student project.

Sources: OpenAI's Booking.com case study on rule-based limits ¹, Google OR-Tools docs on routing ⁶ ⁵, Rasa open-source description ⁴, OpenTripPlanner overview ¹⁰, OpenTripMap API info ⁹, and an Analytics Vidhya guide on an open-source travel chatbot ² ³.

¹ Personalizing travel at scale with OpenAI | OpenAI

<https://openai.com/index/booking-com/>

² ³ Build a Travel Assistant Chatbot

<https://www.analyticsvidhya.com/blog/2024/07/build-a-travel-assistant-chatbot/>

⁴ GitHub - RasaHQ/rasa: Open source machine learning framework to automate text- and voice-based conversations: NLU, dialogue management, connect to Slack, Facebook, and more - Create chatbots and voice assistants

<https://github.com/RasaHQ/rasa>

⁵ OR-Tools | Google for Developers

<https://developers.google.com/optimization>

⁶ ⁷ Vehicle Routing | OR-Tools | Google for Developers

<https://developers.google.com/optimization/routing>

⁸ ⁹ OpenTripMap - Map service for sightseeing and travel planning

<https://opentripmap.com/en/>

10 OpenTripPlanner

<https://www.opentripplanner.org/>